当我们学习一门语言的时候，我们需要关注的是，这门语言有哪些特征（feature），可以应用在哪些场景中使用。

Java的一个特征是Object-Oriented, 即支持class和object。 这一章我们开始学习类(class), 我们会用一系列的例子来展示类给我们带来了哪些方便。

例子1:

写一个程序，让用户输入一个复数z和一个大于0的整数n， 计算如下表达式并输出到屏幕

$$1 + z^1 + z^2 + \ldots + z^n$$

实现1:

对于复数z=a+bi, 复数的运算无非就是a和b的运算，我们可以利用复数运算规则来计算整个表达式的值。对于两个复数$z = a + bi$和$w = c + di$, 我们知道

$$z + w = (a + c) + (b + d)i \qquad z \cdot w = (ac - bd) + (ad + bc)i$$

我们便可以逐个计算出$z^n = z^{n-1} \cdot z$, 再求和。

a) 基本框架

JAVA

```java
import java.util.Scanner;

public class ComplexSeries {
    /** compute the sum of series related to z = a+bi */
    public static double[] sumSeries(double a, double b, double n) {
        double []sum = new double[2];

        return sum;
    }

    public static void main(String[] args) {
        //step 1: interactive part
        System.out.println("Enter the real and imaginary part of z:");
        Scanner input = new Scanner(System.in);
        double x = input.nextDouble(); //the real part
        double y = input.nextDouble(); //the imaginary part
        System.out.print("Enter a postive integer:");
        int n = input.nextInt();

        //step 2: compute the series
        double []sum = sumSeries(x, y, n);
        double realSum = sum[0];
        double imagSum = sum[1];

        //step 3: feedback to the user
        System.out.println("The sum is "+realSum+" + " + imagSum +" I.");
```

```
    }
}
```

b) 业务部分

我们使用函数来计算，因为我们需要返回两个值，所以我们使用数组作为函数的返回值。函数部分的实现如下

JAVA

```
    /**compute the multiplication of two complex number z*w, where z=a+bi,
w=c+di*/
    public static double[] multiply(double a, double b, double c, double d) {
        double []mult = new double[2];
        mult[0] = a*c - b*d;
        mult[1] = a*d + b*c;
        return mult;
    }
    /** compute the sum of series related to z = a+bi */
    public static double[] sumSeries(double a, double b, double n) {
        double []r = {1, 0};
        double []sum = {1, 0};
        for(int i=0;i<n;i++) {
            r = multiply(r[0], r[1], a, b);
            sum[0] += r[0];
            sum[1] += r[1];
        }
        return sum;
    }
```

c)完整代码

JAVA

```
import java.util.Scanner;

public class ComplexSeries {
    /**compute the multiplication of two complex number z*w, where z=a+bi,
w=c+di*/
    public static double[] multiply(double a, double b, double c, double d) {
        double []mult = new double[2];
        mult[0] = a*c - b*d;
        mult[1] = a*d + b*c;
        return mult;
    }

    /** compute the sum of series related to z = a+bi */
    public static double[] sumSeries(double a, double b, double n) {
        double [] r = {1, 0};
```

```
        double []sum = {1, 0};
        for(int i=0;i<n;i++) {
            r = multiply(r[0], r[1], a, b);
            sum[0] += r[0];
            sum[1] += r[1];
        }
        return sum;
    }

    public static void main(String[] args) {
        //step 1: interactive part
        System.out.println("Enter the real and imaginary part of z:");
        Scanner input = new Scanner(System.in);
        double x = input.nextDouble(); //the real part
        double y = input.nextDouble(); //the imaginary part
        System.out.print("Enter a postive integer:");
        int n = input.nextInt();

        //step 2: compute the series
        double []sum = sumSeries(x, y, n);
        double realSum=sum[0];
        double imagSum=sum[1];

        //step 3: feedback to the user
        System.out.println("The sum is "+realSum+" + " + imagSum +" I.");
    }
}
```

运行：

JAVA

```
Enter the real and imaginary part of z:1 0
Enter a postive integer:3
The sum is 4.0 + 0.0 I.
```

JAVA

```
Enter the real and imaginary part of z:0.5 0.5
Enter a postive integer:10
The sum is 1.03125 + 1.0 I.
```

实现2：

我们可以实现一个Complex类来代表复数，并且在其中实现基本的运算。因为我们是我们的第一个class的例子，我们逐步增加其功能。

1) 最基础的Complex class

```java
class Complex{
    double real, imag;
}
```

此时我们可以运行

```java
Complex c = new Complex();
c.real = 1;
c.imag = 10;
System.out.println(c.real+" + " + c.imag + " I");
System.out.println(c);
```

输出结果为

```java
1.0 + 10.0 I
Complex@372f7a8d
```

这里我们可以知道：

1. 我们创建的class可以用new 来创建实例， Complex c定义了一个类型为Complex的变量，并指向new Complex()创建的实例。
2. 在类里声明的变量可以用点(.)来访问，即用点来表示一个实例的属性，这点其实很合理，例如“张三是一个人，他的名字叫张三”这句话在Java中可以这样实现

```java
Person zhangsan = new Person();
zhangsan.name = "张三";
```

3. 直接用println来打印变量c的时候，打印出的是类名+一串16进制的数，其实是内存地址。

2) 添加构造体

刚刚例子里，创建一个新的Complex的实例时并不能同时指定real 和imag。事实上，在java里我们可以使用构造体来实现在创建实例的时候同时初始化real和imag，

```java
class Complex{
    double real, imag;

    Complex(){

    }

    Complex(double a, double b){
        real = a;
```

```
        imag = b;
    }
}
```

此时我们可以这样创建实例

```java
Complex c = new Complex(1,2);
System.out.println(c.real+" + " + c.imag + " I");
System.out.println(c);
```

```java
1.0 + 2.0 I
Complex@372f7a8d
```

构造体有哪些特点：

- 构造体是一种特殊的方法
- 用new 创建实例时调用
- 其名称与类名一致
- 没有返回值，也没有void

需要注意的是，一旦有了有参构造体，我们需要明确写出无参构造体才能保证语句`Complex c = new Complex();`不会报错。

3) 添加toString()方法

```java
class Complex{
    double real, imag;

    Complex(){

    }

    Complex(double a, double b){
        real = a;
        imag = b;
    }

    public String toString() {
        String s = imag >=0 ? " + " : " - ";
        double flag = imag >=0 ? 1 : -1;
        return real+s+flag*imag+" I";
    }
}
```

在添加toString()方法后**（其实是重写，后面会再讲）**，

```java
Complex c = new Complex(2, 10);
System.out.println(c);
```

即可直接打印

```java
2.0 + 10.0 I
```

这是因为在print的时候，会调隐式用toString()方法。

4) 添加add, sub和 mult方法

假如有两个复数$c_1 = 3 + 4i, c_2 = 10 + 20i$, 先需要计算$c_3 = c_1 - c_2$，如果能这样写，会方便很多

```java
Complex c1 = new Complex(3, 4);
Complex c2 = new Complex(10, 20);
//Complex c3 = c1 - c2;
Complex c3 = c1.sub(c2); //c3 = c1 - c2
System.out.println(c3);
```

我们可以这样实现

```java
    Complex sub(Complex w) {
        double r = real - w.real;
        double i = imag - w.imag;
        Complex z = new Complex(r, i);
        return z;
    }
```

`sub`是方法名，`Complex`是方法的返回值，`Complex w`是方法的参数，w.real 是w这个对象的real，`real` 是当前对象的real，也可以写成`this.real`, 如当调用c1.sub(c2) 时，this指的是c1.

添加这些方法后的类可以写成

```java
class Complex{
    double real, imag;

    Complex(){

    }

    Complex(double a, double b){
```

```
        real = a;
        imag = b;
    }

    Complex add(Complex w) {
        double r = real + w.real;
        double i = imag + w.imag;
        return new Complex(r, i);;
    }

    Complex sub(Complex w) {
        double r = this.real - w.real;
        double i = this.imag - w.imag;
        return new Complex(r, i);
    }

    Complex mult(Complex w) {
        double r = real * w.real - imag*w.imag;
        double i = real * w.imag + imag*w.real;
        return new Complex(r, i);
    }

    public String toString() {
        return real + " + " + imag + " I";
    }
}
```

其中

```
Complex z = new Complex(r, i);
return z;
```

这两行合并成了一行，效果一样

```
return new Complex(r, i);
```

5) sumSeries的实现

在Complex的基础上，sumSeries可以简洁一点

```
    public static Complex sumSeries(Complex c, double n) {
        Complex r = new Complex(1,0);
        Complex sum = new Complex(1,0);
        for(int i=0;i<n;i++) {
            r = r.mult(c);
```

```
            sum = sum.add(r);
        }
        return sum;
    }
```

5)完整代码

JAVA

```
import java.util.Scanner;

class Complex{
    double real, imag;

    Complex(){

    }

    Complex(double a, double b){
        real = a;
        imag = b;
    }

    Complex add(Complex w) {
        double r = this.real + w.real;
        double i = this.imag + w.imag;
        Complex z = new Complex(r, i);
        return z;
    }

    Complex sub(Complex w) {
        double r = this.real - w.real;
        double i = this.imag - w.imag;
        return new Complex(r, i);
    }

    Complex mult(Complex w) {
        double r = real * w.real - imag*w.imag;
        double i = real * w.imag + imag*w.real;
        return new Complex(r, i);
    }

    public String toString() {
        return real + " + " + imag + " I";
    }
}

public class ComplexSeries2 {
```

```java
    public static Complex sumSeries(Complex c, double n) {
        Complex r = new Complex(1,0);
        Complex sum = new Complex(1,0);
        for(int i=0;i<n;i++) {
            r = r.mult(c);
            sum = sum.add(r);
        }
        return sum;
    }

    public static void main(String[] args) {
        //step 1: interactive part
        System.out.print("Enter the real and imaginary part of z:");
        Scanner input = new Scanner(System.in);
        double x = input.nextDouble(); //the real part
        double y = input.nextDouble(); //the imaginary part
        System.out.print("Enter a postive integer:");
        int n = input.nextInt();

        //step 2: compute the series
        Complex c = new Complex(x,y);
        Complex sum = sumSeries(c, n);

        //step 3: feedback to the user
        System.out.println("The sum is "+sum);
    }
}
```

运行结果

```
Enter the real and imaginary part of z:0.5 0.5
Enter a postive integer:10
The sum is 1.03125 + 1.0 I.
```

这个例子是复数序列求和问题，这里使用类给出了另一种写法，大家可以通过这个例子来学习类的基本写法。